

Data Mining 2025 - Assignment 1

Bas de Blok, Joris Heemskerk, Alex de Roode

October 17, 2025

Abstract

In recent years, the subfield of supervised learning within machine learning (ML) has expanded significantly. With this expeditious advancement, so has the set of classification algorithms developed by the ML community. This development has replaced the question of whether to utilise machine learning algorithms, with the question of which machine learning algorithms to utilise. Therefore, this study compares five supervised learning algorithms by training the models on a dataset containing 400 true and 400 false negative hotel reviews, with the task being the classification of the authenticity of a given review. The models that fall within the scope of this paper are: Multinomial Naive Bayes (MNB), Logistic Regression (LR), a singular Classification Tree (CT), Random Forest (RF) and Gradient Boosting (GB). The models get compared by answering the following four questions. (1) *How does the performance of the generative linear model (Multinomial Naive Bayes) compare to the discriminative linear model (Logistic Regression)?* (2) *Are Random Forests and Gradient Boosting able to improve on the performance of the linear classifiers?* (3) *Does performance improve by adding bigram features, instead of using just unigrams?* (4) *What are the five most important terms (features) pointing towards a fake review, and what are the five most important terms pointing towards a genuine review?* From our findings we can conclude that the MNB consistently outperforms the LR model. Random forest and Gradient Boosting do not outperform the linear models, despite their more complex nature. Additionally, when adding bigram features, provided improvements across all metrics. The most important features were also found and reported upon in the conclusion.

1 Introduction

As the prevalence of the internet and online platforms has grown in the past twenty years, so has the influence of online reviews of hotels in guiding consumer's purchasing decision. However, the trustworthiness of those reviews is not what it once was. As of 2025 it is not a rare occurrence to witness review sections filled with questionable and presumably fake reviews, with many online platforms doing little to stop the spread of such misleading reviews.

Developments within machine learning have led to the development of large set of algorithms suitable for predicting the truthfulness of such reviews. Therefore, a straight-forward solution would be to utilize such algorithms to separate true from false reviews. However, there are many different classification algorithms, each with their own inductive biases, leading to differences in performance. Therefore, selecting the algorithm that would perform best within this specific context is not as straight-forward. This paper aims to compare five different classification algorithms in order to give a holistic perspective five algorithms perform within the context of predicting the truthfulness of hotel reviews, and juxtaposes their underlying performative differences.

Firstly, we analyse Multinomial Naive Bayes [?], a model using probability estimates from the dataset to generate predictions. Secondly, we evaluate Logistic Regression [?], a model that linearly combines features from data to fit a predictive model. Thirdly, we analyse Classification Trees [?], who use a tree-like structure to cluster and classify data. Fourthly, we expand upon these Classification trees with Random Forest classifiers [?], who use multiple classification trees to improve performance. Lastly, we use Gradient Boosting [?], another expansion on classification trees, gradually building them on each other to achieve higher accuracy.

Each algorithm gets trained on a dataset consisting of genuine and fake reviews, tuning the hyperparameters for optimal balance between predictive performance and computational simplicity. After evaluating every model, comparisons get drawn, pointing out differences, biases, strengths and weakness for each model, relative to one another.

In general, this research aims to answer the following four questions. (1) *How does the performance of the generative linear model (Multinomial Naive Bayes) compare to the discriminative linear model (Logistic Regression)?* (2) *Are Random Forests and Gradient Boosting able to improve on the performance of the linear classifiers?* (3) *Does performance improve by adding bigram features, instead of using just unigrams?* (4) *What are the five most important terms (features) pointing towards a fake review, and what are the five most important terms pointing towards a genuine review?*

2 Data

The data which the mentioned algorithms are trained upon, comes from a dataset of genuine and fake hotel reviews, constructed by Myle Ott and others [?, ?]. The dataset consists of 1600 reviews, split evenly into four categories: positive truthful, positive fake, negative truthful and negative fake. However, this study only concerns itself with the reviews carrying a negative sentiment; those being the negative truthful and negative fake reviews. Therefore this study only works with the 800 reviews belonging to that subset, instead of the entire dataset. Furthermore, these 800 reviews are provided as 5 equally sized sets of 160, four of which will be used as the training dataset and one as the validation data. This validation data will not be touched until the experiments described in Section 3.6.

As is usual in data-science/machine-learning projects: data first has to be preprocessed before being used to train models. Knowing the data consists of hotel reviews, it is evident that the dataset originally came in the unstructured form of raw text. Therefore, significant preprocessing was needed to funnel the data into a representation that sufficed for training classification algorithms. This happened in three stages, those being general data cleaning, then a bag-of-words conversion, and finally the removal of sparse terms. Each stage gets explained below.

General Data Cleaning In this first stage, all the words got converted to lowercase. After this step, the data got simplified by removing punctuation, symbols, white-spaces and numbers. Once that step concluded, stop words that appeared frequently and conveyed minimal information got removed. Words such as: "the", "a", "an", and more. These steps ensure removal of semantically uninteresting information. The information removed does not contribute to the predictive power of each datapoint.

Bag-of-Words Once the data had been cleaned, we transformed the text-data into a bag-of-words representation. This converts the text data into a numeric representation where every unique word in the training partition becomes a feature and the value of this feature per review becomes the frequency of each word. This transforms the unstructured text data into a more structured alternative that prediction models can reason over. However, this decision also came with its disadvantages. After all, converting text data into a bag-of-words representation does bring with it some information loss due to word order within individual reviews getting erased.

Here we split our dataset into two. For one part we exclusively look at single word frequencies: unigrams. The other part also includes combinations of two words: bigrams. For example, the datapoint "hello world" contains unigrams "hello" and "world", as well as the bigram "hello world". At the end of this step, the dataset including only unigrams contains 6,956 features, whereas the dataset including both unigrams and bigrams contains 49,392 features. Again, it is important to note that these features are exclusively sampled from the training partition, the validation partition will not be used to generate features. Instead it will be fitted to the features found in the training partition.

Removal of Sparse Terms Most machine learning models do not perform optimally with a very large set of features. We therefore reduce the feature space by removing sparse terms. These are unigrams and or bigrams that appear in only a small part of the dataset. For the data partition without bigrams, we decided to remove the bottom 1% of words, as indicated by their relative frequency in the training partition. This left 350 features with the least frequent feature appearing in 32 out of 640 reviews. For the data partition including bigrams, we opted to increase the percentage to the bottom 5% of words, by relative frequency. This left 1621 features with the least frequent occurring in 7 of 640 reviews. We reflect on this decision in Section 6.

3 Algorithms

This section describes delves into more detail regarding all five models, for both the dataset with exclusively unigrams as well as the dataset containing both unigrams and bigrams. In total 10 experiments will be described, each aiming to find the optimal hyperparameter configurations for each model individually. After which the best found configurations for each model will be compared in the final section.

3.1 Multinomial Naive Bayes

First we analysed the multinomial naive Bayes classifier (MNB) [?]. It is often used in text classification tasks, such as sentiment analysis. MNB is a probabilistic classifier, meaning it predicts the class of an input based on the probability distribution of the classes given the observed data. MNB assumes that features are independent within each class, which means that one feature does not affect the probability of another feature occurring. This assumption reduces the amount of computation needed since it allows the joint probability of all features to be expressed as the product of individual probabilities. MNB estimates the probability of each class, as found in the dataset, and predicts the class with the highest probability, as

$$C_{\text{NB}} = \arg \max_{c \in C} P(c) \prod_{i=1}^m P(x_i | c),$$

where m represents the number of features that describe one input, x_i the value of feature i , and C the set of possible classes.

A problem occurs when an MNB is used on data it has not seen before in its training data. When this happens $P(x_i | c)$ will be 0 for this feature while predicting which results in the entire prediction being 0 for all classes. To avoid the problem of zero probabilities for features that do not appear in the training data for a given class, MNB uses smoothing. Smoothing adjusts the estimated probabilities by adding a small constant to the feature counts, ensuring that no probability is set to zero. The most common form is Laplace smoothing [?], where a constant α is added to all feature counts. The standard Laplace smoothing is when $\alpha = 1$. With smoothing, the estimated probabilities within each class is calculated as

$$\hat{P}(x_i | c) = \frac{\text{count}(x_i, c) + \alpha}{\sum_{x_j \in V} \text{count}(x_j, c) + |V|},$$

where $\text{count}(x_i, c)$ is the number of times feature x_i occurs in class c and V the vocabulary of the train set.

This α is a hyperparameter of MNB's. A small α value results in less smoothing, which may lead to overfitting when rare features dominate a class. On the other side, a large α can lead to underfitting by making the model less sensitive to the actual feature distribution.

To ensure the reliability of model comparisons, all models were evaluated using 10-fold cross-validation on the training partition. In this process, the training data is divided into ten folds (subsets). Each fold is once used as a validation set, while the other nine folds are used for training the model. As such we can evaluate the model by the average accuracy across all folds. Accuracy was chosen as the evaluation metric as it's a straightforward measure of representing the amount of correctly classified data.

For hyperparameter selection, we used the one standard error (1-SE) rule. This rule selects the simplest model whose mean accuracy over all folds lies within 1-SE of the model with the highest mean accuracy. By doing so, we favour models that generalise better.

To find the best value for alpha we chose to analyse the value $\{0.1\}$ and the values within the range $[2.5, 40]$ with increments of 2.5, combining both a small fractional value and a set of larger values. This range was stopped at 50 since less than 10% of the features occur more than fifty times in the trainingsdata. Each α was evaluated with the cross-validation setup as described above on the dataset excluding bigrams and on the dataset including bigrams.

3.2 Logistic Regression with Lasso Penalty

Next we analysed the Logistic Regression classifier (LR) [?], a linear model that estimates the probability of an instance belonging to a particular class. For binary classification, LR models the relationship

between the input features and the log-odds of the probability of the class as a linear combination of the features. Applying the sigmoid function to this linear combination maps any input to a value between zero and one, allowing the model to output probabilities. This can be expressed as

$$P(Y = 1 | \mathbf{x}) = \frac{1}{1 + e^{-\beta^\top \mathbf{x}}},$$

where $\mathbf{x}$ is the feature vector and $\beta^\top$ the weight vector. The model learns these weights by minimising a loss function based on the log-likelihood function of the observed data.

To prevent overfitting and improve generalisation, we use regularisation by adding a penalty term for the size of the coefficients to the objective function. With Lasso penalty. This encourages sparsity in the weight vector by driving less important feature weights towards zero. The regularized loss function can be calculated as

$$E(\beta) = -l(\beta) + \lambda \sum_{j=1}^m |\beta_j|,$$

where $E(\beta)$ is the new error function that we want to minimize, $-l(\beta)$ is the new negative log-likelihood function and m is the number of features. Here λ controls the strength of the regularisation. In practice, most implementations use the hyperparameter $C = \frac{1}{\lambda}$, meaning that smaller values for C mean stronger regularisation.

To find the best value for C we chose to analyse the values we first performed a broad search over the values $\{0.0001, 0.001, 0.01, 0.1, 1, 10, 100, 1000, 10000\}$. This provided insight into the scale at which regularisation had a meaningful effect. The results differed greatly between the models trained on only unigrams and the models also trained on bigrams. The results for models trained on unigrams indicated that the model performance was best between $C = 0.1$ and $C = 0.5$. Afterwards we performed a search using small linear steps within the range $[0.125, 2.5]$ with increments of 0.125 to more precisely find the region of best performance.

The results for models also trained on bigrams indicated that the model performance was best between $C = 1000$ and $C = 10000$. Afterwards we performed a search using steps within the range $[1000, 10000]$ with increments of 1000. Each C was evaluated using the cross-validation and model selection method as described in section 3.1.

3.3 Single Classification Trees

In addition to the MNB and LR models, we also trained a classification tree (CT) [?]. CT's work by repeatedly splitting the dataset on specific features, until the split group meets a specific criteria, such as all sharing the same class label. Each split group is called a node, and each node that is not split any further is called a leaf node. In order to evaluate which split on a feature is best, there are multiple metrics to consider, common ones being Resubstitution error [?], Gini-index [?], and Entropy [?]. For our research, we opted to use the Gini-index, as this was not only the default option in Python, but also because of its computational efficiency and it being the dominant metric from our Data mining lectures.

Additionally, the shape and performance of a tree are determined by some (hyper)parameters, such as the maximal depth of the tree, the minimum samples per node in order to consider splitting it, or the minimum samples per node before it turns into a leaf node by default. However, for our research, we do not aim to tune any of these parameters, instead opting to grow an unconstrained tree and then pruning it afterwards. Pruning, here, means reducing the complexity of the tree in such a way that predictive power loss on the training partition of the dataset is minimised. For this, we use the Cost-Complexity Pruning algorithm as described by Breiman et al. [?]. The algorithm minimises the cost $C_\alpha(T)$ of a tree T using

$$C_\alpha(T) = R(T) + \alpha |\tilde{T}|,$$

where $R(T)$ is the resubstitution error, $|\tilde{T}|$ the number of leaves, and α the penalty for the complexity of the tree. This means α decides the severity to which we reduce the complexity of the tree. Using Cost-Complexity Pruning, a nested sequence of pruned subtrees can be calculated, such that all trees in this sequence minimise cost $C_\alpha(T)$ for an increasing range of α values. This sequence can be written as

$$T_{max} > T_1 > T_2 > T_3 > \dots > \{t_1\},$$

where T_{max} is the maximally grown tree, and t_1 is the single root node shared by all trees. Each T_k in this sequence is the smallest minimising subtree for $\alpha \in [\alpha_k, \alpha_{k+1})$.

In order to prune a single tree on the train set, we would first train the maximally unconstrained tree T_{max} from the training partition of the dataset. We would then compute the pruning sequence $T_{max} > \dots > \{t_1\}$. For each T_k in the sequence, we compute the accuracy score on the test sample, and select the preferred tree. Usually, the one standard error (1-SE) rule is applied, selecting the tree with the highest accuracy and then finding the smallest minimising subtree within 1-SE from the pruning sequence (which is the one with the highest α).

Here, the configurations are different α values, for which the smallest minimising subtree is trained. These different α values are obtained from the pruning sequence of T_{max} , trained on the entire training partition, resulting in $\alpha_1, \alpha_2, \dots, \alpha_k$. Instead of simply using these values, we compute a new sequence $c_1, c_2, \dots, c_k$, where $c_1 = 0$, $c_k = \infty$, and $c_{k-1} = \sqrt{\alpha_{k-1} - \alpha_k}$. These values c are used as α in cross-validation. We apply the same cross-validation and model selection method as described in section 3.1.

3.4 Random Forests

Single CT's are not always the most effective predictors because they tend to overfit and have high variance, making their predictions unstable and sensitive to small changes in the data. One algorithm that aims to improve the predictive performance of CT's is the Random Forest algorithm (RF) [?]. RF's use CT's as their building blocks, but instead of training a single tree, RF's draw multiple subsets from the data (with replacement) and train a different tree on every subset. This procedure is called bootstrapping. For prediction, the majority vote of all these trees is used as the classification. The number of these subsets, and therefore trees/estimators, is a hyperparameter of RF's.

Another difference between RF's and CT's comes down to splitting nodes. Instead of selecting the best split based on every feature, RF's only looks at a subset of the features. This helps remove unwanted data correlations from the tree. The size of this subset is also a hyperparameter.

Since this is the first model that has two hyperparameters, we applied a grid search. All combinations of both hyperparameters are set up against each other, and the combination with the highest accuracy on the training partition (again using cross-validation) is chosen as the best model. Regarding the number of trees, we chose to use the range [10,150], with step size 10. We also include a tree with size 1, which is the simplest forest model. This range is chosen arbitrarily, providing a relatively limited search space, which is computationally feasible given our time and resource constraints. For the size of the feature subset for each split, we chose to analyse some commonly found values: $\{\sqrt{\cdot}, \log_2, 0.3, 0.5, 0.7, 1.0\}$, where each is a factor of the number of features present. This range should provide a good overview of the bias-variance trade-off. Combined these hyperparameters result in a grid of 96 experiments. With the same cross-validation and model selection method as described in section 3.1, this means 1920 trained RF's.

3.5 Gradient Boosting

RF's are just one way of looking at using multiple trees to improve classification performance. Another way is using the Gradient Boosting algorithm (GB) [?]. This algorithm also uses an ensemble of trees to predict, but instead of growing trees on part of the data and features, it grows small trees sequentially. Each tree is grown using information from the previous tree, specifically by focussing on the errors (or residuals) made by the previous tree. This way, each subsequent tree corrects for the mistakes of the previous, gradually improving over-all accuracy. The main difference in the resulting trees, is their size. Most trees in GB's are only a few levels deep. The maximal depth of the trees can be tuned as a hyperparameter, with common values being 1 or 3 for classification. In our preliminary tests, we found no significant differences for different maximal depth values. Therefore, we do not tune this parameter beyond the default value of 3.

Additionally, since GB's are built sequentially, correcting errors over time, a learning rate is used to determine how quickly to correct the error. Common values for this hyperparameter are 0.01 and 0.001. For our research we used values $\{0.00001, 0.0001, 0.001, 0.01, 0.1, 1\}$. This should provide a wide enough range to see the effect of the learning rate on the performance of the models.

Lastly, the number of trees grown is another important hyperparameter. For these, we decided to analyse a larger range of [20,300], with step size 20 (as well as the model with only 1 tree). This is

the same number of steps as in FR’s, but with a wider range as GB’s learn incrementally and may therefore need more trees to increase accuracy. Combined with the same cross-validation and model selection method as described in section 3.1, we again end up with 1920 trained models.

3.6 Comparing Models

Finally, after tuning every individual model, we have to select the best performing model out of the five selected. We separate this task between the two datasets, with and without the bigrams. For each dataset, we train the 1-SE model, as these models provide the best trade-of between simplicity and predictive performance. Each configuration gets trained on the entire training partition and validated on the test partition we kept separated from the rest for all steps until now. For each model, we compute the accuracy, precision, recall, and F1 scores.

3.7 Important Features

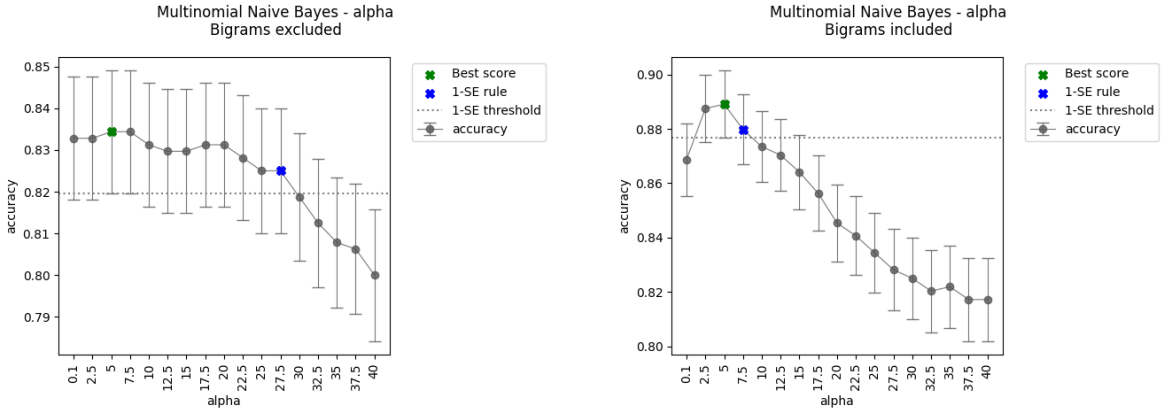
After selecting the best performing, we perform an analysis to identify which features contribute the most to the classification. We will find the five most important features pointing toward a fake review, and the five most important features pointing toward a genuine review. We will only do so for the best model.

4 Results

This section reports the results from the experiments described in Section 3. For each model, the results regarding hyperparameter tuning are reported, followed by the comparison between models.

4.1 Multinomial Naive Bayes

This section contains the results of the trained multinomial naive bayes models. The results are split between a version using only unigrams in Figure 1a and a version also using bigrams in Figure 1b. The best performing model in Figure 1a scores an accuracy of 83.44% with $\alpha = 5$. The 1-SE rule model here scores an accuracy of 82.5% with $\alpha = 27.5$. In the model including bigrams (Figure 1b), the best performing model scores an accuracy of 88.91% with $\alpha = 5$. The 1-SE rule scores an accuracy of 87.97% with $\alpha = 7.5$.



(a) Hyperparameter tuning of α for MNB’s on the dataset including only unigrams.

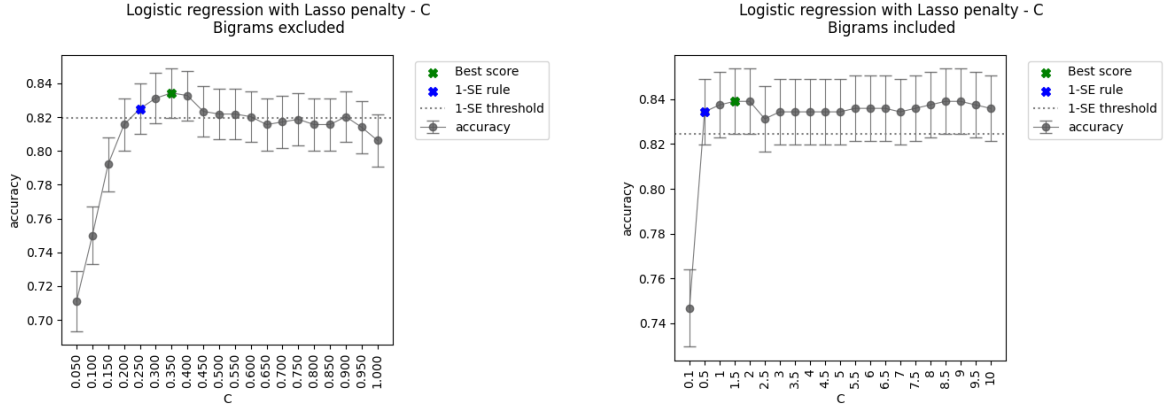
(b) Hyperparameter tuning of α for MNB’s on the dataset including bigrams.

Figure 1: Comparison of hyperparameter tuning of α for multinomial naive bayes.

4.2 Logistic Regression with Lasso Penalty

This section contains the results of the trained logistic regression with Lasso penalty models. The results are split between a version using only unigrams in Figure 2a and a version also using bigrams

in Figure 2b. The best performing model in Figure 2a scores an accuracy of 83.59% with $C = 0.375$. The 1-SE rule model here scores an accuracy of 82.19% with $C = 0.5$. In the model including bigrams (Figure 2b), the best performing model scores an accuracy of 84.53% with $C = 2000$. The 1-SE rule scores an accuracy of 83.13% with $C = 8000$.



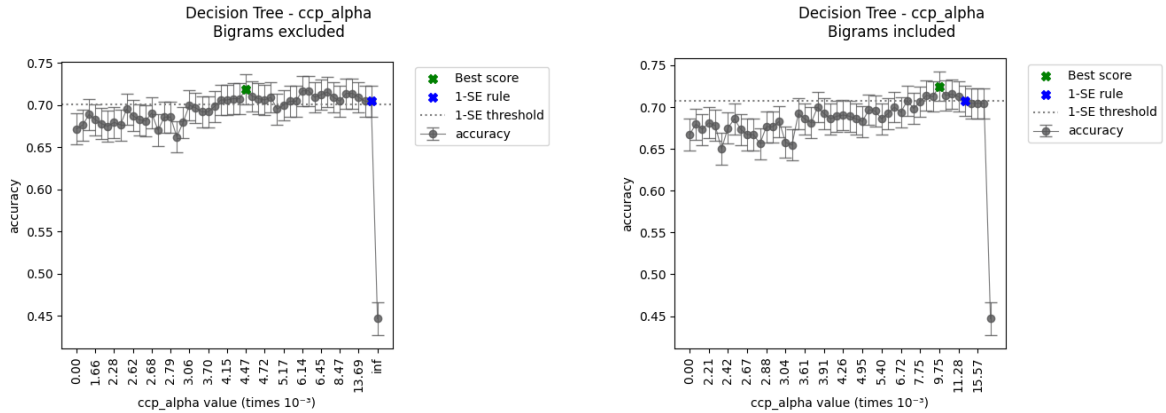
(a) Hyperparameter tuning of C for LR's on the dataset including only unigrams.

(b) Hyperparameter tuning of C for LR's on the dataset including bigrams.

Figure 2: Comparison of Hyperparameter tuning of C for logistic regression.

4.3 Single Classification Trees

Again, we provide the results of the Cost-Complexity Pruning of our classification trees, using cross-validation on the training partition. The results are split between a version using only unigrams in Figure 3a and a version also using bigrams in Figure 3b. The best performing model in Figure 3a scores an accuracy of 71.72% with $\alpha \approx 4.279 \times 10^{-3}$. The 1-SE rule model here scores an accuracy of 70.47% with $\alpha \approx 36.33 \times 10^{-3}$. In the model including bigrams (Figure 3b), the best performing model scores an accuracy of 72.50%, with $\alpha \approx 9.75 \times 10^{-3}$. The 1-SE rule scores an accuracy of 70.78%, with $\alpha \approx 12.31 \times 10^{-3}$.



(a) Cost-Complexity Pruning of classification trees on the dataset including only unigrams.

(b) Cost-Complexity Pruning of classification trees on the dataset including bigrams.

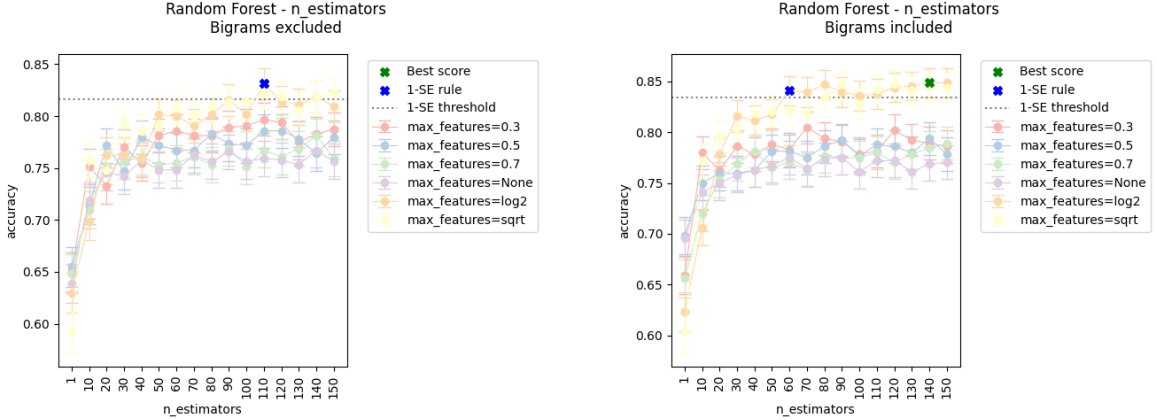
Figure 3: Comparison of Cost-Complexity Pruning.

4.4 Random Forests

This section contains the results of the trained Random Forest models. As stated, we performed a grid search comparing the number of estimators and the number of features to consider when evaluating

a split. In Figure 4a, we provide an overview of these parameters for the dataset excluding bigrams. Here, we see the use of the square root as the maximum number of features to consistently outperform most other values, only sometimes being surpassed by $\log_2$. Increasing the number of estimates has a positive effect on predictive performance, which starts to slowly stagnate from around 40 and onwards. The best performing model here is equal to the 1-SE model and has an accuracy of 83.13%, with $\text{max_features} = \log_2$ and $\text{n_estimators} = 110$.

The same patterns emerge for the models including bigrams in Figure 4b, with main difference being a larger performative gap between $\log_2$ and $\sqrt{\cdot}$, and the others. Here, the best performing model has an accuracy of 84.84%, with $\text{max_features} = \log_2$ and $\text{n_estimators} = 140$. The 1-SE rule model has an accuracy of 84.06%, with $\text{max_features} = \log_2$ and $\text{n_estimators} = 60$.



(a) Grid search of the number of estimators and the number of features per split for RF's on the dataset including only unigrams.

(b) Grid search of the number of estimators and the number of features per split for RF's on the dataset including bigrams.

Figure 4: Grid search results for Random Forests.

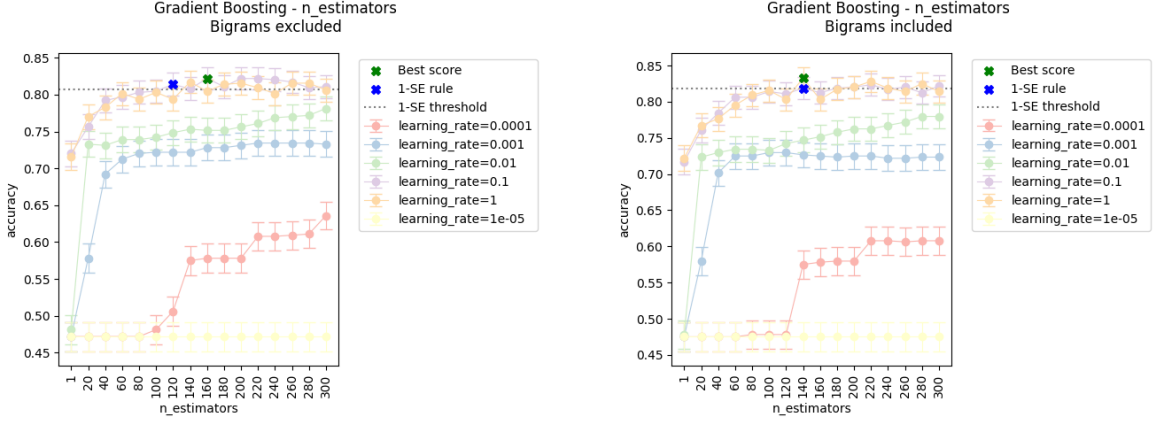
4.5 Gradient Boosting

This section contains the results of the trained Gradient Boosting models. As stated, we performed a grid search comparing the number of estimators and the learning rate. In Figure 5a, we provide an overview of these parameters for the dataset excluding bigrams. We can see a large effect of the learning rate on the predictive performance. For example a learning rate of 0.0001 seems to need more than 300 trees to achieve competitive performance with the other learning rates. Larger learning rates seem to converge earlier, with both 0.1 and 1 achieving the highest accuracies. The number of trees for these models does not seem to matter much beyond 20 or 40. In this figure, the best scoring model achieves an accuracy of 82.19%, with $\text{learning_rate} = 0.1$ and $\text{n_estimators} = 160$. The 1-SE model has an accuracy of 81.40%, with $\text{learning_rate} = 0.1$ and $\text{n_estimators} = 120$.

The same patterns can be found in the models including bigrams in Figure 4b. Here, the best scoring model achieves an accuracy of 83.28%, with $\text{learning_rate} = 1$ and $\text{n_estimators} = 140$. The 1-SE model has an accuracy of 81.88%, with $\text{learning_rate} = 0.1$ and $\text{n_estimators} = 140$.

4.6 Comparing Models

After selecting the best hyperparameter configurations using the 1-SE rule, we compared all models on the separate test partition. Results can be found in Tables 1 and 2. It can be seen that for both datasets, the Multinomial Naive Bayes outperforms the others on the test partition. Except for precision, where it gets outperformed by Logistic Regression on the unigram only dataset and by Random Forests on the bigrams dataset.



(a) Grid search of the number of estimators and the learning rate for GB's on the dataset including only unigrams.

(b) Grid search of the number of estimators and the learning rate for GB's on the dataset including bigrams.

Figure 5: Grid search results for Gradient Boosting.

Table 1: Validation results for unigram only models. Per numeric column, the best results are highlighted in bold.

Model	Accuracy	Precision	Recall	F1-score
MNB	0.806	0.788	0.818	0.803
LR	0.769	0.813	0.747	0.778
DT	0.638	0.650	0.634	0.642
RF	0.788	0.800	0.780	0.790
GB	0.750	0.800	0.727	0.762

4.7 Important Features

As shown in Section 4.6, the best performing model on both our datasets is MNB. With MNB we found the most important features by determining which terms contributed the most to the classification. We calculated the feature importances by examining the log probabilities of each feature for both classes. For each feature, we calculated the difference between its log probability for fake and genuine reviews to obtain a measure of how indicative it is for each class.

Based on this analysis, the top five features that indicate a fake review in our unigram only MNB model were *{recently, decided, chicago, smell, luxury}*, while for our MNB model including bigrams they were *{millennium, hotel chicago, chicago hotel, smell, luxury}*.

The top five features that indicate a genuine review in our unigram only MNB model were *{elevators, location, great, star, open}*, while for our MNB model including bigrams they were *{elevators, th floor, priceline, security, cool}*.

Table 2: Validation results for models including bigrams. Per numeric column, the best results are highlighted in bold.

Model	Accuracy	Precision	Recall	F1-score
MNB	0.869	0.900	0.847	0.873
LR	0.812	0.800	0.821	0.810
DT	0.650	0.750	0.625	0.682
RF	0.856	0.912	0.820	0.864
GB	0.762	0.812	0.739	0.774

5 Conclusion

In Section 1, we conclude with four research questions. Below, we will answer them in order, starting with (1) *How does the performance of the generative linear model (Multinomial Naive Bayes) compare to the discriminative linear model (Logistic Regression)?* When looking at the cross-validation results from both MNB and LR, we find no big difference in terms of accuracy for best performing models on the unigrams only dataset (83.44% for MNB to 83.59% for LR). The same is true for the 1-SE models (82.5% to 82.19%). For unigrams and bigrams the difference increased to 88.91% and 84.53% between the best performing models of MNB and LR, respectively. For the 1-SE models, the difference is similar, being between 87.97% and 83.13%. When looking at the results on the test set, from Section 4.6, we find much larger differences between MNB and LR in most metrics. MNB consistently wins, with the only exception being the Precision on the unigram only dataset, where MNB gets beaten by LR 78.8% to 81.3%.

(2) *Are Random Forests and Gradient Boosting able to improve on the performance of the linear classifiers?* While the Classification Trees consistently get outperformed by both RF and GB, this is not the case for both NMB and LR. In the results from Section 4.6, we find close competition between RF, GB, and LR, with MNB being slightly further ahead than all. In most metrics, LR wins it from both RF and GB, but not always, such as it losing from RF in Accuracy for unigram only models, and RF winning in precision on the bigram dataset. In general, based on our results we cannot conclude that RF and GB are able to improve on the performance of linear classifiers.

(3) *Does performance improve by adding bigram features, instead of using just unigrams?* As seen in the results from Section 4.6, we consistently find almost all metrics to increase when adding bigrams. Except for the Recall of our DT, which decreases from 63.4% to 62.5%. However, with this being a minor decrease and only one instance, we conclude performance to increase by adding bigrams in the case of our research.

(4) *What are the five most important terms (features) pointing towards a fake review, and what are the five most important terms pointing towards a genuine review?* As seen in the results from Section 4.7, we determined the most important features by using the log probabilities of our MNB model for unigram only and for including bigrams. The top five features that indicate a fake review in our unigram only MNB model were {*recently* *decided* *chicago* *smell* *luxury*}, while for our MNB model including bigrams they were {*millennium* *hotel chicago* *chicago hotel* *smell* *luxury*}. The top five features that indicate a genuine review in our unigram only MNB model were {*elevators* *location* *great* *star* *open*}, while for our MNB model including bigrams they were {*elevators* *th floor* *priceline* *security* *cool*}.

6 Discussion

For this research, a lot of decisions were made when setting up the experiments. Namely the specific hyperparameter values to analyse. While our methods are clearly described and rigorously followed, they may lack grounding. For example, the level of detail with which we looked at the hyperparameters. Explaining why we did not take more, and or smaller steps or increased or decreased our range is easy, but making sure the explanation holds is not. For future research we would like to better ground our hyperparameter ranges such that we can provide better and more consistent conclusions. However, we do think our conclusions would not change much given a different hyperparameter tuning approach.

In addition to the hyperparameter tuning, we also realise our approach to selecting the best model might not be optimal. We relied on the 1-SE rule for our statistical testing method. There may exist better approaches. We chose to use the 1-SE rule primarily because it was repeatedly discussed and used in the lectures.

Further more, we would like to reflect on the model selection. We were limited by the assignment to exclusively analyse the five models used. We do however think other models would achieve higher performance. Especially Natural Language based models. For future research we recommend analysing the performance of other models, as we expect significant performance increases.

Lastly we reflect on our decision made in Section 2, where we chose to drop 1% and 5% of the data for unigrams, and unigrams and bigrams, respectively. We neglected to report our method, as we found it to be rather unsubstantiated. We came to these values by copying the lecture material, rather than finding a grounded reasoning ourselves. For this problem, these values gave us results that

roughly matched our expectations. In future research, we recommend finding a more grounded reason or performing a small empirical study to find the best values.

7 Use of Generative AI

This research was almost exclusively made without the use of Generative AI (Gen-AI). Gen-AI was only used to elaborate on theory already provided by lectures, but where we needed more clarity and examples to make it stick. We also asked Gen-AI on occasion if our ideas and executions were considered valid given its knowledge base (as a sort of sanity check). We also used Gen-AI a hand full of times to suggest or correct our writing, making sure to only validate single sentences or parts of sentences as to make sure all of the work remains ours and such that it remains within our cognitive grasp.